MASTER PROJECT DOCUMENT / 10 NOVEMBER 2026

Flipkart AI Electronics Advisor

Project Architecture & Five-Phase Development Plan

A practical implementation and handoff guide for a five-member team.

PROJECT	DELIVERY MODEL
RAG-powered electronics shopping assistant	Five ordered phases, reviewed in one shared GitHub repository
Python + Pandas + Sentence Transformers	ChromaDB + LLM API + Streamlit

Build answers from product evidence. Make every phase reproducible. Hand over work with clear acceptance.

What is preserved
The architecture, selected dataset, technology choices, all phase tasks, technical parameters, deliverables, acceptance criteria, rubric and demo structure come from the referenced conversation. Team assignments, integration contracts and formal handoff procedures are added guidance.

Document map

Pages	Contents
2-4	Scope, stack, architecture and product-data contracts
5-9	The original five-phase plan, with phase handoff checks
10-12	Team responsibilities, repository structure and GitHub workflow
13-14	Formal handoff process and reusable template
15-16	Evaluation, grading, presentation and source notes

Prepared from “Explain RAG Assignment” and the attached technical assignment brief. Planning document only; no dataset inspection or application implementation has been performed.

Purpose, stack and scope

Build a **RAG Chatbot** that understands customer requirements, searches an electronics product database, compares suitable products and recommends options supported by actual product information. Prioritize correct RAG implementation, reliable answers and a smooth live demonstration.

Expected customer-facing behavior

- Recommend electronics based on requirements and budget.
- Answer specification questions, compare products and explain why they match the customer's needs.
- Show which dataset records support the answer; refuse to invent unavailable information.

Selected data source

Kaggle Flipkart Products - PromptCloudHQ. The original plan identifies 20,000 products across multiple categories, with `description`, `product_specifications`, `brand` and `discounted_price` among the relevant fields. Inspect the CSV first to determine which electronics records are usable. Start with approximately **150-300 high-quality electronics records**, subject to actual coverage.

Open the selected Kaggle dataset

Component	Technology / reason
Programming / data	Python / Pandas: one language for the application and CSV processing
Embeddings	`all-MiniLM-L6-v2` via Sentence Transformers: lightweight local model
Vector database	ChromaDB: simple, persistent vector search
LLM	OpenAI API or another available LLM: grounded explanations
Interface	Streamlit: required by the assignment
Evaluation / version control	Python + CSV test cases / GitHub

Use a **modular Python application**, with direct integrations; LangChain is not needed initially. Avoid unnecessary microservices.

Submission scope

Essential: Clean electronics data, embeddings, ChromaDB, semantic retrieval, LLM integration, grounded answers, Streamlit, testing and failure analysis.

If time permits: Hybrid search, budget filtering, source display, comparison formatting and example questions were listed as improvements in the original scope summary. Budget filtering and source preservation are also explicit phase tasks; retain them in this delivery plan. Hybrid RAG remains conditional on feasibility and course coverage.

Not needed: Authentication, payments, live inventory integration, LLM fine-tuning, microservices or a custom frontend. Historical dataset prices and product facts must never be presented as current Flipkart stock or prices.

02 / SYSTEM DESIGN

Two pipelines, one application

Data preparation runs when creating or rebuilding the product database. The query pipeline runs for each customer message.

A / PREPARE THE DATA

Kaggle CSV dataset
Original product information

↓

Clean + filter electronics
Duplicates, empty descriptions, irrelevant rows

↓

Product documents + chunking
Name, description and specifications

↓

Embedding model
all-MiniLM-L6-v2 / Sentence Transformers

↓

Persistent ChromaDB
Embeddings + source metadata

B / ANSWER A QUESTION

Streamlit user question
Suggest a laptop under INR 50,000

↓

Query understanding
Product type, needs and numeric constraints

↓

ChromaDB retriever
Semantic search + exact-price filter

↓

LLM + retrieved context
Generate only from supported evidence

↓

Grounded answer
Recommendations, comparisons, sources or fallback

How the modules connect

Process	Modules and responsibility
Offline preparation	`preprocess.py` cleans CSV data; `chunking.py` prepares documents; `embeddings.py` creates vectors; `vector_store.py` persists them; `scripts/build_index.py` orchestrates ingestion.
Online answering	`app.py` collects input; `retrieval.py` fetches evidence; `rag_pipeline.py` combines evidence, prompt and LLM; `prompts.py` controls grounding.
Evaluation	CSV questions and results in `data/evaluation/`; retrieval and RAG checks in `tests/`.

Evidence boundary
The LLM receives the user question plus retrieved product context and source IDs. It must not invent specifications, price, availability or future launch dates. If the dataset cannot support an answer, return an honest insufficient-evidence fallback.

Preserve product identity

The sample schema and parameter values below preserve the original plan. The integration contract is added guidance so five people can develop compatible modules.

Cleaned record: original illustrative structure

```
{
  "product_id": "P001",
  "name": "Example Laptop",
  "category": "Laptops",
  "brand": "Example Brand",
  "price": 49999,
  "description": "Laptop with 16GB RAM...",
  "specifications": "16GB RAM, 512GB SSD"
}
```

Illustrative structure only. These are not verified product details. Preserve original source fields or a source-row mapping alongside the cleaned schema.

Original retrieval parameters

Setting	Starting point
Record selection	Approximately 150-300 high-quality electronics records, subject to availability
Chunking	Keep short product records intact. For long text, start with 500-800 characters and 75-character overlap; test context preservation.
Embedding / store	Sentence Transformers `all-MiniLM-L6-v2` / persistent ChromaDB
Retrieval	Approximately 5 relevant results per question; preserve product name, ID, price and source for every chunk.
Budget constraints	Extract numeric limits such as “under ₹50,000” and apply numeric filtering where possible.

Added contract for team integration

- Each chunk has a stable `chunk_id`, `product_id`, product name, category, normalized numeric price when known, and original source reference. Never detach specifications from their product identity.
- Use the same embedding model and configuration for indexing and queries. Record chunk settings, model identifier and collection name in the index build metadata.
- The retriever returns evidence text plus source metadata. The RAG pipeline returns answer text plus the supporting product/source IDs for the UI.
- Unknown or invalid prices remain explicitly unknown. They must not be treated as zero or claimed to meet a budget. Repeated ingestion must not silently duplicate indexed chunks.

PHASE 1 / FOUNDATION

Project foundation and dataset engineering

Objective: Create a clean, trustworthy electronics dataset.

Accountable lead: M1 - Data & Foundation | **Receiving lead:** M2 - Retrieval & Index

Implementation tasks - original plan

01 Initialize the GitHub repository and Python environment.
02 Download and inspect the selected Kaggle dataset.
03 Identify electronics categories and usable product records.
04 Remove duplicates, empty descriptions and irrelevant records.
05 Normalize prices, brands, product categories and specifications.
06 Select approximately 150-300 high-quality electronics records, subject to what the dataset contains.
07 Create a standardized description for each product while preserving factual details.
08 Record the original dataset source and limitations.

Deliverables and acceptance

Deliverables: Cleaned dataset, data dictionary, project repository and documented use case.

Acceptance: Every retained product has a usable description and unique identifier. Enough real electronics data exists to support meaningful retrieval.

Phase handoff checklist - added

- [ ] Record raw/cleaned row counts, electronics categories, exclusion reasons, missing fields and duplicate counts.
- [ ] Provide the data dictionary, stable IDs, source mapping and a reproducible cleaning command.
- [ ] M2 loads the processed data and confirms usable descriptions and numeric-price handling. If usable electronics coverage is insufficient, log the issue and resolve the dataset decision before indexing.

Complete `docs/handoffs/phase-01.md` using the template on page 14. Attach evidence, merge the reviewed work, and record receiver acceptance before advancing.

Embeddings, chunking and vector database

Objective: Make the product database searchable by meaning, rather than only matching exact words.

Accountable lead: M2 - Retrieval & Index | **Receiving lead:** M3 - Grounded RAG

Implementation tasks - original plan

01 Convert product records into structured text documents.
02 Keep short product records together; split lengthy descriptions into chunks if necessary.
03 Start with approximately 500-800-character chunks and 75-character overlap for lengthy text; test whether these preserve context.
04 Generate text embeddings using Sentence Transformers.
05 Store embeddings and source metadata inside ChromaDB.
06 Implement semantic similarity search to retrieve approximately 5 relevant results per question.
07 Verify that retrieving a chunk preserves its product name, ID, price and source.

Deliverables and acceptance

Deliverables: Persistent ChromaDB index, data-ingestion script and working retrieval function.

Acceptance: Questions such as “laptop for students” return relevant records. Retrieved evidence can be inspected before anything is sent to the LLM.

Phase handoff checklist - added

- [ ] Save embedding model, chunk settings, index/collection location, input dataset identity and rebuild command.
- [ ] Show sample queries with top-five evidence and product/source metadata; verify identity remains attached to specifications.
- [ ] M3 rebuilds or opens the index, runs retrieval independently and checks that repeated ingestion creates no duplicate chunks.

Complete `docs/handoffs/phase-02.md` using the template on page 14. Attach evidence, merge the reviewed work, and record receiver acceptance before advancing.

AI reasoning, grounded answers and safeguards

Objective: Connect retrieval to an LLM and make the chatbot answer reliably.

Accountable lead: M3 - Grounded RAG | **Receiving lead:** M4 - Streamlit & Integration

Implementation tasks - original plan

01 Connect a selected LLM API.
02 Create a controlled prompt that restricts answers to retrieved evidence.
03 Include product details and source IDs in the context passed to the LLM.
04 Implement product recommendations and comparisons.
05 Extract budget constraints such as “under ₹50,000” and apply numeric filters.
06 Prevent unsupported specifications, prices and availability claims.
07 Implement an insufficient-evidence fallback.
08 Add minimal chat history without allowing older messages to override retrieved evidence.

Deliverables and acceptance

Deliverables: End-to-end RAG answer pipeline, prompt templates and unsupported-question handling.

Acceptance: The assistant responds with dataset-supported answers and does not confidently invent missing information.

Phase handoff checklist - added

- [ ] Record provider/model, configuration variable names, prompt version and answer/source return format; never include API key values.
- [ ] Demonstrate a recommendation, comparison, budget query and unsupported question, with retrieved context and source IDs.
- [ ] M4 runs the pipeline outside the UI and confirms sources, fallback behavior and minimal chat-history handling.

Complete `docs/handoffs/phase-03.md` using the template on page 14. Attach evidence, merge the reviewed work, and record receiver acceptance before advancing.

Original illustrative expected behavior
Question: “Suggest a laptop under ₹50,000 with 16GB RAM.” Answer: “Example Laptop; Price: ₹49,999; RAM: 16GB; Storage: 512GB SSD; Source: Product P001 in the sample dataset.” This uses the sample record only, not a verified product.

PHASE 4 / FUNCTIONAL APPLICATION

Streamlit application and RAG enhancement

Objective: Turn the working RAG system into a polished, easy-to-demonstrate chatbot.

Accountable lead: M4 - Streamlit & Integration | **Receiving lead:** M5 - Evaluation & Release

Implementation tasks - original plan

01 Build the Streamlit chat interface.
02 Add conversation history and clear-chat functionality.
03 Show product recommendations with prices and specifications.
04 Display retrieved sources alongside AI answers.
05 Add helpful example question prompts.
06 Show user-friendly loading and error states.
07 Implement Hybrid RAG as the preferred optional enhancement, if feasible and covered in class.
08 Test the interface on the machine used for the presentation.

Deliverables and acceptance

Deliverables: Complete Streamlit chatbot and optional hybrid-retrieval improvement.

Acceptance: Users can ask questions, receive grounded responses, inspect supporting records and recover gracefully from errors.

Phase handoff checklist - added

- [ ] Supply launch/setup instructions, configuration variable names and a working end-to-end UI walkthrough.
- [ ] Check history, clear-chat, sources, loading, API/index errors and historical-data wording on the presentation machine.
- [ ] If Hybrid RAG is included, document keyword + semantic result merging/ranking and compare it with baseline retrieval; otherwise record why it was deferred.

Complete `docs/handoffs/phase-04.md` using the template on page 14. Attach evidence, merge the reviewed work, and record receiver acceptance before advancing.

Original enhancement and interface concept
Hybrid RAG combines semantic similarity with keyword matching, useful for exact model numbers. The proposed UI reads “Flipkart AI Advisor”, “What are you shopping for?”, “Discover and compare electronics with AI”, and “Ask about electronics...”. This is an illustrative interface, not an existing app.

PHASE 5 / FINAL ACCEPTANCE

Testing, evaluation, optimization and presentation

Objective: Prove the chatbot works and prepare the demonstration required by the assignment.

Accountable lead: M5 - Evaluation & Release | **Receiving lead:** All five members / final release

Implementation tasks - original plan

01 Prepare approximately 12-20 test questions, exceeding the assignment minimum of 3-4.
02 Include factual lookup, comparison, multi-part and unsupported questions.
03 Evaluate retrieval relevance and whether answers match the evidence.
04 Measure retrieval success, for example whether the correct product appears in the top five retrieved results.
05 Document at least two actual failure cases, their causes and proposed fixes.
06 Fix issues in chunking, metadata, filtering or prompts.
07 Prepare the project README and architecture explanation.
08 Rehearse the complete five-minute group demonstration.

Deliverables and acceptance

Deliverables: Tested chatbot, evaluation results, documented failure cases, presentation material and reproducible setup instructions.

Acceptance: The app works end to end, all required question types are tested, two failure cases are documented and the live demo fits five minutes.

Phase handoff checklist - added

- [ ] Attach question-level results, top-five retrieval evidence, answer checks and at least two observed failure analyses.
- [ ] Re-run affected checks after fixes; record unresolved limitations and a reproducible setup on the presentation machine.
- [ ] All five members accept the submission tag, README, presentation script, demo timing and final limitations.

Complete `docs/handoffs/phase-05.md` using the template on page 14. Attach evidence, merge the reviewed work, and record receiver acceptance before advancing.

Five people, shared delivery

Assign actual names to M1-M5 at kickoff. One member leads each phase; the others contribute continuously. A lead owns integration and acceptance evidence, rather than completing an entire phase alone.

Member	Primary responsibility	Ongoing contribution
M1 / Data & Foundation	Dataset source, cleaning, schema, provenance, data dictionary; Phase 1 lead.	Explain data limits; support data-related test failures; review changes affecting facts.
M2 / Retrieval & Index	Chunking, embeddings, ChromaDB, retrieval, metadata; Phase 2 lead.	Tune retrieval; support numeric filters and optional hybrid search; review indexing changes.
M3 / Grounded RAG	LLM integration, prompts, evidence-only answers, comparisons, fallback; Phase 3 lead.	Budget extraction and history logic; review answer correctness and unsupported-query handling.
M4 / Streamlit & Integration	Chat UI, history, clear-chat, source display, loading/errors; Phase 4 lead.	Maintain app integration and presentation-machine setup; support demo flow.
M5 / Evaluation & Release	Test corpus, results, failure analysis, reproducibility, final demo; Phase 5 lead.	Set up issue board and review checks early; maintain README and handoff completeness.

Who contributes in each phase

Phase	Lead	Support and receiving review
1 / Data	M1	M2 validates schema for ingestion; M3 checks answerable facts; M4 outlines UI inputs; M5 records data-quality evidence.
2 / Retrieval	M2	M1 resolves data issues; M3 verifies evidence contract; M4 inspects source fields; M5 adds retrieval cases.
3 / Grounding	M3	M2 supports filtering; M1 validates facts; M4 integrates the answer format; M5 checks fallback and hallucinations.
4 / App	M4	M3 integrates responses; M2 supports sources/hybrid; M1 checks labels; M5 tests full user flows.
5 / Release	M5	M1-M4 fix issues in their areas, review evidence, rehearse and approve the final submission.

Coordination rhythm
Hold a short daily sync: completed work, next task and blockers. Keep one phase milestone active, with a GitHub issue for each task. Each phase closes with a walkthrough and independent reproduction by its receiving lead. No deadline was supplied; schedule dates after Phase 1 confirms data availability.

Repository layout

The original project structure is retained below. Items marked “added” support team handoffs and repeatable review. Generated data and indexes need a documented storage/rebuild policy.

```
flipkart-ai-advisor/
  app.py                        # Streamlit interface
  requirements.txt              # Python dependencies
  .env.example                  # Configuration names/placeholders
  README.md                     # Setup and documentation
  .gitignore                    # added: secrets/generated files
  data/
    raw/                        # Original Kaggle CSV
    processed/                  # Clean electronics records
    evaluation/                 # Test questions and results
  src/
    preprocess.py               # Clean dataset
    chunking.py                 # Prepare documents
    embeddings.py               # Generate embeddings
    vector_store.py             # ChromaDB setup
    retrieval.py                # Retrieve products
    rag_pipeline.py             # LLM and grounding
    prompts.py                  # Prompt templates
  scripts/
    build_index.py              # Create searchable database
  tests/
    test_retrieval.py
    test_rag.py
  docs/
    architecture.md
    evaluation.md
    presentation.md
    data_dictionary.md          # added
    handoffs/                   # added
      TEMPLATE.md
      phase-01.md ... phase-05.md
  .github/                      # added
    pull_request_template.md
    ISSUE_TEMPLATE/
    workflows/checks.yml
```

Storage and configuration rules - added

- Keep API keys in environment variables or Streamlit secrets, never in GitHub. `.env.example` contains variable names and dummy values only. Ignore `.env`, `.streamlit/secrets.toml`, local virtual environments, caches and generated ChromaDB files.
- Document the exact dataset download source, filename and cleaning/index commands. Track only data that may be redistributed under its terms; otherwise provide download instructions and a checksum. Store generated indexes outside version control and rebuild them from documented inputs.

Review, merge and release

Use a simple feature-branch workflow with one shared `main` branch. Keep `main` runnable. No separate development branch is necessary for this project.

Step	Action / evidence
1 / Plan	Create milestones Phase 1-5. Every issue states its owner, acceptance criteria, dependencies and phase. Board columns: Backlog, Ready, In progress, In review, Done.
2 / Branch	Create a short branch from current `main`, e.g. `feat/p2-chroma-index`, `fix/p3-budget-filter` or `docs/p4-handoff`. Keep each branch focused on one issue.
3 / Implement	Commit coherent changes. Record setup/configuration changes and update documentation alongside the code. Coordinate shared-file edits, particularly `app.py`, `requirements.txt` and retrieval contracts.
4 / Open pull request	Link the issue; explain behavior, setup/data changes, checks run and limitations. Include retrieved evidence or UI images when relevant. A teammate reviews the change before merge.
5 / Validate	Run relevant data, retrieval or RAG checks. Run secret-free automated checks for PRs; manually run LLM-dependent and UI checks with local credentials and attach results. Do not require API secrets for every review.
6 / Merge	Resolve feedback and conflicting changes; run affected checks again. Squash merge after one non-author review and passing required checks. Delete the merged branch.
7 / Close phase	Complete `docs/handoffs/phase-0N.md`, link PRs and evidence, and obtain receiver acceptance. Tag the accepted snapshot `phase-1-complete` through `phase-5-complete`.
8 / Submit	From the accepted Phase 5 snapshot, tag `v1.0.0`. Record the commit, launch instructions, evaluation results and final known limitations.

Pull request checklist

- [ ] Linked issue and phase; concise description of the resulting behavior.
- [ ] Relevant verification and evidence attached; no credentials or unsupported product claims.
- [ ] README, configuration example and handoff updated when affected.
- [ ] Reviewer confirms contracts and independently runs the relevant flow.

Protection and exceptions
Enable protection for `main` with pull requests, one review and required automated checks where repository settings allow. A blocking defect keeps the phase open; an accepted nonblocking limitation needs an issue, owner, impact and next step. Do not record an unrun check as passed.

Handoff is an acceptance event

Every phase produces a written handoff, a reviewed code snapshot and a short live walkthrough. Sending a document alone does not close the phase. The receiving lead must reproduce the key result.

Universal handoff checklist

- [ ] Scope: phase objective, completed tasks, deferred items and acceptance criteria are explicit.
- [ ] Snapshot: merged PRs, exact commit, phase tag and working branch are recorded.
- [ ] Artifacts: dataset/index/configuration versions, module paths, source mapping and output locations are linked.
- [ ] Reproduction: environment, dependency installation, data preparation, index build, launch and check commands are exact and tested. Never include secret values.
- [ ] Validation: expected versus actual results, representative evidence and all failed/unrun checks are attached.
- [ ] Limitations: unresolved issues list severity, impact, owner and proposed fix; data gaps and historical-price limits are visible.
- [ ] Contract: input/output schema, function behavior and configuration changes are communicated to the next phase.
- [ ] Transition: receiving lead completes an independent smoke test and acknowledges the result; next tasks and owners are agreed.
- [ ] Approval: outgoing lead, receiving lead and QA reviewer record decision, date and commit. Phase 5 requires acceptance by all five members.

Handoff meeting agenda

Activity	What must be shown
Walkthrough	Explain completed scope, changed modules, data/index configuration and important decisions.
Reproduction	Receiving lead follows written steps from a clean checkout or documented clean setup, then runs the phase-specific checks on pages 5-9.
Decision	Accept, accept with tracked nonblocking issues, or return for changes. Identify every blocker and its owner.
Next phase	Confirm the receiving owner, interfaces and first actionable tasks. Close the milestone only after accepted checks and documentation are merged.

Formal gate rule
Critical failures in source identity, reproducibility, retrieval, grounding, numeric constraints or the current phase objective block acceptance. Scope changes require a recorded decision and updated checks. No phase is marked complete solely because its code was merged.

Copy for every phase

Save as `docs/handoffs/phase-0N.md`. Fill every field; use “Not applicable” with a reason where needed. Add links to evidence rather than pasting large logs.

Field	Complete with
Identity	Phase [1-5] / title / handoff date / outgoing lead / receiving lead / QA reviewer
Version	Repository URL / merged PRs / exact commit / phase tag / environment and dependency versions
Objective and scope	Objective / tasks completed / deferred scope with issue links / decisions and rationale
Artifacts and contracts	Changed paths / dataset source and fingerprint / raw-cleaned-chunk counts / schema / model ID / chunk and overlap settings / collection / function input-output changes
Configuration	Required variable names and how to set them / local secret storage / provider and model; never include key values
Reproduction	1. Checkout the commit 2. Install the pinned dependencies 3. Obtain and clean the documented data 4. Rebuild/open the index 5. Run checks and launch the app Replace each step with the exact verified command and expected output.
Results	Check ID / question or action / expected behavior / actual result / PASS, FAIL or NOT RUN / evidence link
Issues and risks	Issue ID / severity / observed impact / workaround / owner / next action. Distinguish blockers from nonblocking limitations.
Next-phase package	Receiving objective / first three tasks with owners / required inputs / known interface constraints
Acceptance and sign-off	Decision: Accept / Accept with tracked issues / Changes required Independent reproduction: who / when / result / evidence Outgoing lead: name, date, commit Receiving lead: name, date, commit QA reviewer: name, date, commit Phase 5: acceptance from M1, M2, M3, M4 and M5

Acceptance evidence table - repeat as needed

Check / criterion	Expected	Actual + evidence	Status / owner
[Phase-specific check]	[Observable result]	[Result and link]	[PASS / FAIL / NOT RUN; name]
[Receiver smoke test]	[Can reproduce independently]	[Environment and log]	[Decision; name]

The phase-specific checks on pages 5-9 supplement this template. Record results from actual runs; the example text above is a blank template, not completed evidence.

Prove retrieval and grounding

The original plan targets **12-20 questions**, above the assignment minimum of **3-4**. Use factual lookups, comparisons, multi-part questions and unsupported requests; document at least **two actual failure cases**.

Original example test	Expected behavior
Find a specific product	Retrieve the correct record.
Recommend under ₹50k	Respect price limits.
Compare two laptops	Use the correct specifications.
Ask about an unknown product	Clearly state insufficient data.
Ask about future availability	Avoid inventing a date.

Added evaluation record and measures

Each CSV row should record question ID, question type, expected product/source IDs, constraints, retrieved IDs, answer, citations, pass/fail notes and run configuration. Base questions on the actual cleaned records; examples mentioning laptops do not prove that suitable laptop records exist.

- **Retrieval success@5:** For tests with an expected product, count a success when that product appears in the top five retrieved results. Report successes / eligible tests and the rate. For multi-part cases, also check whether all required evidence was retrieved.
- **Answer checks:** Correctness, relevance, faithfulness to context, valid product citations and numeric-budget compliance. Unsupported questions should acknowledge missing evidence; report them separately from answerable retrieval tests.
- **Two failure analyses:** Capture the actual question, expected behavior, retrieved evidence, observed answer, cause, proposed fix and retest result. Do not invent failures to fill the report.
- **Optional enhancement:** If Hybrid RAG is implemented, compare the same questions against the baseline and record how keyword/vector results are merged or ranked.

Original assessment weights

Assessment	Weight	Project focus
Problem definition and dataset quality	15%	Phase 1
RAG pipeline: chunking, embeddings, vector store, retrieval, generation	50%	Phases 2-3 + Phase 4 enhancement
Grounded answers and handling unsupported questions	15%	Phase 3
Streamlit UI and usability	10%	Phase 4
Testing, evaluation and error analysis	10%	Phase 5

Spend the most effort on retrieval and grounded generation: together they account for 65% of the suggested rubric. Fix data, chunking, metadata, filters and prompts based on observed results.

Five-minute demo and release

The assignment requires a group of 4-5 students, a working Streamlit RAG chatbot and a live presentation capped at **five minutes**. The original recommended split below keeps the overlapping brief timings within that limit.

Time	Speaker / action - added allocation
0:00-1:00 / Problem	M1 introduces Flipkart, consumer electronics, the target shopper, dataset source and historical-data limits.
1:00-2:00 / Demo	M2 shows one factual lookup and its retrieved source evidence.
2:00-3:00 / Demo	M3 shows a budget recommendation or comparison and explains grounding.
3:00-4:00 / Demo	M4 shows a multi-part or unsupported request, fallback and source display.
4:00-5:00 / Q&A	M5 opens Q&A; each member answers questions about their area. Keep answers brief and finish by 5:00.

Final release checklist - added

- [ ] All five phase handoffs are accepted and linked from the README.
- [ ] Fresh setup can obtain data, clean it, rebuild the index and launch Streamlit.
- [ ] Evaluation results and two actual failure cases are complete; unresolved limits are explicit.
- [ ] Demo questions were verified against the real dataset and rehearsed on the presentation machine.
- [ ] The final tag and commit are recorded; all five members can explain the complete pipeline.

Source and fidelity notes

Primary plan: “Explain RAG Assignment”, referenced conversation ID `6af3288b-8a20-83e8-aa51-03b2dd2ba5c8`; architecture and five-phase response, dataset recommendation and assignment explanation retrieved on 10 November 2026.

Assignment brief inspected: “RAG_Chatbot_Assignment_Technical (1) (1).docx”, attached to that conversation. It confirms the baseline RAG requirement, conditional advanced technique, meaningful descriptions, Streamlit, testing, two failure cases, group size, rubric and strict five-minute limit.

Preserved choices: Python/Pandas, local `all-MiniLM-L6-v2`, ChromaDB, an LLM API, Streamlit, approximately 150-300 products, 500-800-character long-text chunks with 75-character overlap, approximately five retrieved results, 12-20 test questions and the original five phases in order.

Conditional scope: Hybrid RAG is the preferred enhancement if feasible and covered in class. Query Decomposition RAG remains the alternative mentioned in the assignment; it is not added to the baseline. If no suitable public data is available, the brief permits a reviewed, clearly labelled synthetic dataset with at least 50 records and meaningful descriptions. This is a contingency, not a replacement for the selected Flipkart dataset.

Added guidance: Team allocation, contracts, repository support files, GitHub review/release steps, measures and formal handoff procedures are recommendations for executing the preserved plan. Dataset suitability, exact LLM/provider, dates and final member names must be settled during implementation.